

INSAT – National Institute of Applied Science and Technology

Basic Compiler

Architecting a 3-Phase Compiler Pipeline

4th Software Engineering 2025-2026

Youssef Fadhloun
Mohamed Ali Guerfali

Iyed Mdimegh
Ahmed Jammoussi

Project Overview & Architecture

This project implements a complete front-end compiler pipeline for a custom programming language, backed by a PyQt5 graphical user interface.

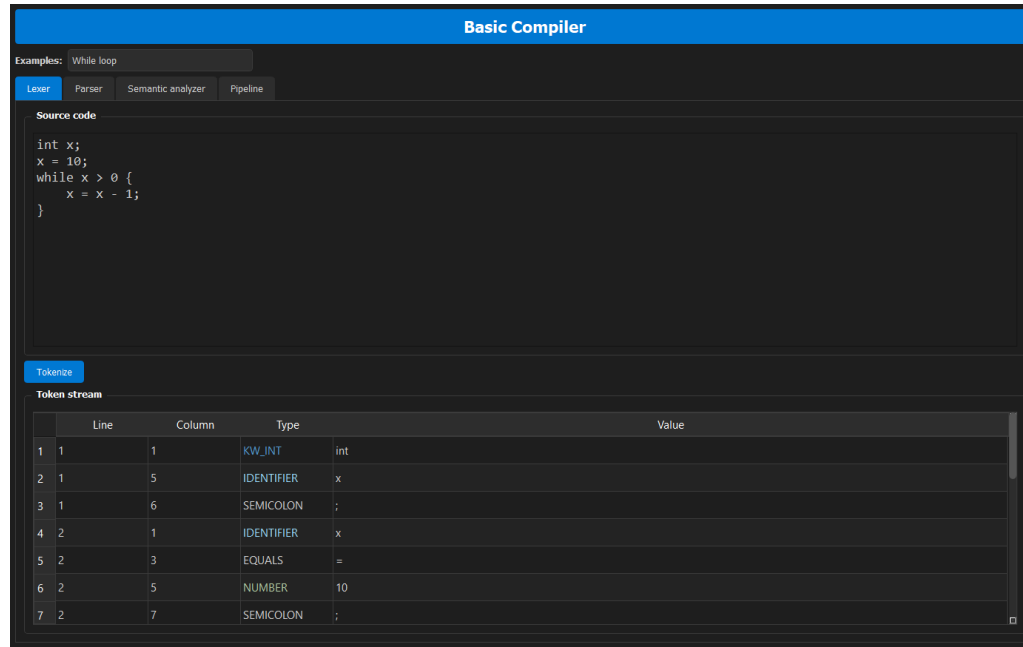
- **Language Specifications:** Supports typed variables (`int`, `string`), arithmetic/comparison expressions, `if/then/else` conditionals, and `while` loops.
- **Unified Architecture:** - A central `Compiler` facade class chains the `Lexer`, `Parser`, and `Semantic Analyzer`.
 - A shared `ErrorHandler` collects errors independently across all phases without immediately halting compilation.
- **Technology Stack:** Python 3.12, PyQt5, built entirely from scratch (No external parser generators like `PLY` or `Lark`).

Phase 1: Lexical Analysis

The lexer converts raw source code into a typed stream of tokens using a custom finite-state approach.

- **Regex-Priority Engine:** Iterates through source text using a priority-ordered list of pre-compiled regular expressions (`TOKEN_PATTERNS`).
- **Keyword Disambiguation:** Strict left-to-right matching ensures reserved keywords (e.g., `int`, `while`) are matched before general identifiers.
- **Token Vocabulary:** Defines 23 distinct terminal categories (Keywords, Operators, Literals, Delimiters, and an EOF sentinel).
- **Positional Tracking:** Every token object firmly tracks its (`line`, `column`) origin, powering precise error reporting downstream.

Lexer Visualization



The screenshot shows a web-based GUI for a 'Basic Compiler'. At the top, there's a blue header with the text 'Basic Compiler'. Below it, a navigation bar contains tabs for 'Lexer' (which is active), 'Parser', 'Semantic analyzer', and 'Pipeline'. Under the 'Lexer' tab, there's a 'Source code' section with a text area containing the following code:

```
int x;  
x = 10;  
while x > 0 {  
    x = x - 1;  
}
```

Below the source code, there's a 'Tokenize' button. Underneath the button is a 'Token stream' section containing a table with 5 columns: 'Line', 'Column', 'Type', and 'Value'. The table displays the tokens generated from the source code.

	Line	Column	Type	Value
1	1	1	KW_INT	int
2	1	5	IDENTIFIER	x
3	1	6	SEMICOLON	;
4	2	1	IDENTIFIER	x
5	2	3	EQUALS	=
6	2	5	NUMBER	10
7	2	7	SEMICOLON	;

Figure 1: GUI Lexer Tab showing the typed Token Stream with precise positional tracking

Phase 2: Syntax Analysis

The parser validates grammatical structure and outputs an Abstract Syntax Tree (AST).

- **Recursive-Descent:** Hand-written, top-down parser mapping CFG non-terminals to specific `parse_*` methods.
- **Mathematical Precedence:** Enforced directly via the method call chain:
`parse_expression` → `parse_comparison` → `parse_addition` → `parse_multiplication` → `parse_unary` → `parse_primary`.
- **Error Recovery:** The `expect(token_type)` utility consumes expected tokens or logs errors safely, allowing parsing to continue and gather multiple syntax errors per run.
- **Node Decoupling:** Every AST node inherits from an `ASTNode` base and implements a `accept(visitor)` method.

AST Graphical Visualization

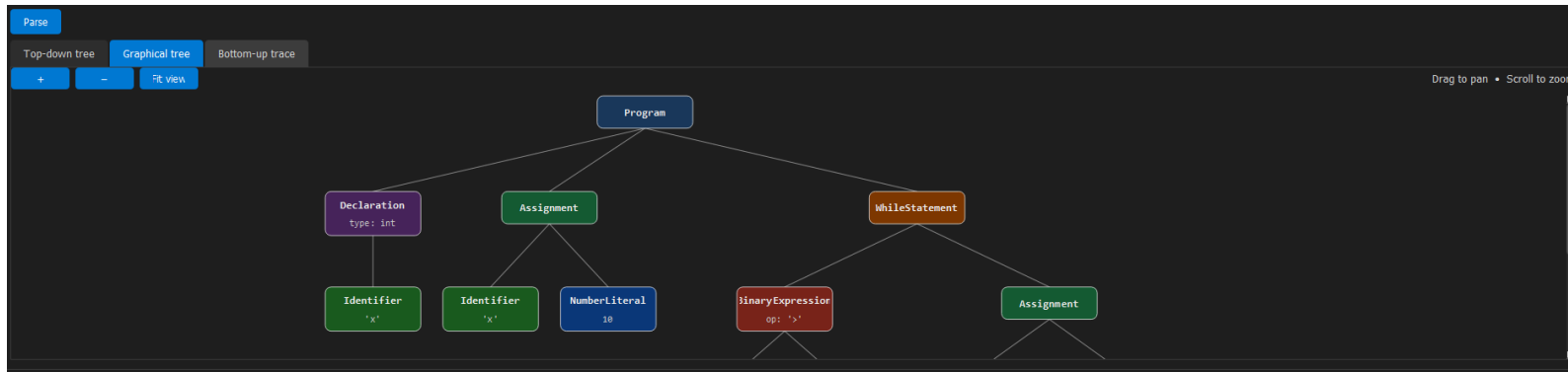


Figure 2: Interactive GUI Canvas displaying the typed AST structure

Bottom-Up LR Trace Simulation

To provide pedagogical value, the compiler simulates an LR shift-reduce sequence despite using a Top-Down parser.

- **Post-Hoc Emulation:** A dedicated module walks the successfully generated AST.
- **Action Generation:** - Emits one **SHIFT** action for every leaf token encountered.
 - Emits one **REDUCE** action (annotated with the grammar rule) as the traversal exits an interior node.
- **Interactive Playback:** The GUI provides a step-by-step animated trace table with **Prev**, **Next**, and **Auto-play** controls to demonstrate bottom-up compilation.

Animated Parsing Trace

Parse

Top-down tree

Graphical tree

Bottom-up trace

◀ Prev

Next ▶

▶ Auto-play

◻ Reset

Step 0 / 33

Speed:

■ SHIFT

■ REDUCE

■ ACCEPT

	Step	Action	Symbol	Stack	Lookahead	Rule / Note
	29	REDUCE	stmt	int primary stmt... }	stmt	→ IDENTIFIER = expr ;
	30	SHIFT	}	int primary stmt... \$		
	31	REDUCE	stmt	int primary stmt... \$	stmt	→ while expr { block }
	32	REDUCE	program	int primary stmt... \$	program	→ statement*
	33	ACCEPT	\$	int primary stmt... \$		Input accepted

Figure 3: Shift-Reduce simulation post-hoc derived from the AST

Phase 3: Semantic Analysis

The semantic analyzer ensures logical correctness, scope validation, and type safety.

- **Scoped Symbol Table:** Implemented as a stack of dictionaries. `enter_scope()` and `exit_scope()` are triggered at block boundaries (e.g., `if`, `while`) to support lexical scoping and variable shadowing.
- **Visitor Pattern Traverse:** A `SemanticAnalyzer` traverses the AST dynamically. For instance, `visit_binary_expression` recursively checks sub-expression types.
- **Strict Rule Enforcement:**
 - Rejects undeclared variables or duplicate declarations.
 - Prevents type mismatches (e.g., `string` assigned to `int`).
 - Validates arithmetic (e.g., preventing subtraction between strings).

Semantic Error Catching

Examples: Type mismatch error

Lexer Parser **Semantic analyzer** Pipeline

Source code

```
int x;  
x = "hello";
```

Analyze

1 semantic error(s) found.

Symbol table

	Scope	Name	Type	Line	Column
1	Scope 0	x	int	1	5

Semantic errors

	Line	Column	Error
1	2	1	Type mismatch: cannot assign string to int

Figure 4: Semantic Analyzer catching a Type Mismatch and undeclared variables

Testing Architecture

The compiler is validated against a comprehensive automated test suite to ensure stability across edge cases.

- **End-to-End Suite:** Written using `pytest`, executing the full `Compiler` pipeline.
- **Test Categories:**
 - **Valid Programs:** Confirms AST structure and zero-error runs on `arithmetic.src`, `conditional.src`, etc.
 - **Invalid Programs:** Intentionally feeds semantic errors (`type_mismatch.src`, `undeclared_var.src`) to ensure the `ErrorHandler` catches and formats exact failure messages.
- **Real-Time GUI Highlighting:** Built-in `QSyntaxHighlighter` provides instant visual feedback in the IDE.

Thank you for your attention!